

Grundlagenpraktikum: Rechnerarchitektur

Praktikumsordnung

1. Grundsätzliches

Diese Praktikumsordnung soll einen ordnungsgemäßen Ablauf des Grundlagenpraktikums „Rechnerarchitektur“ (GRA) gewährleisten. Sie ist nicht Bestandteil der Prüfungsordnung und gibt auch keine Hinweise auf den fachlichen Inhalt des Praktikums.

2. Durchführungsform**2.1. Form des Praktikums**

Das Praktikum wird in Form von Gruppenprojekten durchgeführt. In der ersten Hälfte des Semesters werden im Praktikum und durch Hausaufgaben die notwendigen Grundlagen vermittelt. Die hier erarbeiteten Kenntnisse werden für die Projektphase vorausgesetzt.

Im zweiten Teil des Semesters erhält jede Gruppe (in der Regel drei Personen) ein Projekt, das innerhalb einer vorgegebenen Frist zu bearbeiten ist. Für das erfolgreiche Bestehen des Praktikums sind die Projektimplementierung und die Projektpräsentation Voraussetzung.

Die Projektpräsentation ist grundsätzlich in *deutscher Sprache* vorzutragen. Unter Umständen kann eine Präsentation in englischer Sprache erlaubt werden, wenn die Praktikumsleitung dies bekannt gibt. In diesem Fall ist die Präsentation in englischer Sprache zu halten und die Folien und der Kurzbericht ebenfalls in englischer Sprache zu erstellen.

2.2. Prüfungsvoraussetzungen und Notenbonus

Als Teilnehmer des Praktikums haben Sie keine formalen Voraussetzungen für die Projektphase. Die Hausaufgaben sind freiwillig.

Studierende, die aufgrund ihrer Fachprüfungsordnung 8 ECTS für dieses Praktikum benötigen, müssen eine Zusatzaufgabe erfolgreich absolvieren (Abschnitt [2.6](#)). Studierende, die nur 5 ECTS benötigen, sind von dieser Aufgabe befreit.

Bei Erreichen von mindestens 75 % der Hausaufgabenpunkte gibt es einen Notenbonus (siehe Abschnitt [4.2](#)).

2.3. Vorgehensweise

Die Zuordnung der Gruppen und Aufgaben erfolgt durch Praktikumsleitung¹. Die Projekte sind wie die Hausaufgaben über Artemis abzugeben. Die Projekte werden dort über ein Versionsverwaltungs-System (Git) bereitgestellt. In diesem sind auch regelmäßig Zwischenstände zu speichern, die eine eindeutige Zuordnung der Urheberschaft der verschiedenen Teile der Implementierung über das *Author*-Feld von Commits ermöglichen. Die dort im *README.md* vorgegebene Ordnerstruktur und erwarteten Dateinamen sind einzuhalten, insbesondere auch Groß- und Kleinschreibung. Für die Bewertung wird der letzte Commit des *main*-Branches zum Zeitpunkt der Abgabefrist herangezogen.

2.4. Dauer der Bearbeitung

Das Praktikum ist in einem Semester zu absolvieren. Mit Ausgabe der Projektaufgaben wird ein Abgabetermin festgelegt, zu dem das Ergebnis eingereicht werden muss. Sollte die Frist nicht eingehalten werden können, ist rechtzeitig ein Antrag auf Fristverlängerung zu stellen. Die Praktikumsleitung entscheidet in Rücksprache mit dem Tutor, ob eine Verlängerung gewährt wird. Eine zweite Fristverlängerung ist ausgeschlossen. Wird der Antrag nicht genehmigt und das Projekt trotzdem nach Fristende eingereicht, gilt das Praktikum als nicht bestanden.

2.5. Verhalten der Gruppenmitglieder

Es wird erwartet, dass jedes Gruppenmitglied an allen Projektteilen mitarbeitet und zu jedem Teil Fragen beantworten kann. Die Gruppe hat nachzuweisen, dass sie sich um die Integration aller Mitglieder bemüht hat. Sollten dennoch Gruppen mit weniger als drei Personen zustande kommen, so gelten die Ausnahmeordnungen unter Punkt 5.1.

2.6. Zusatzleistung für 8 ECTS

Studierende, die auf Grund ihres Studienplans 8 ECTS benötigen, müssen in Einzelarbeit eine Zusatzleistung erbringen, die nicht Bestandteil der Projektarbeit ist. Form und Zeitraum der Zusatzaufgabe werden gesondert bekannt gegeben. Bewertet wird lediglich, ob die Anforderungen vollständig erfüllt wurden, eine darüber hinausgehende qualitative Bewertung erfolgt nicht. Studierende, die lediglich 5 ECTS benötigen, müssen die Zusatzleistung nicht erbringen.

3. Projektanforderungen

Das Praktikumsprojekt besteht aus mehreren Teilen: der Implementierung und ihrer Analyse, und der Projektpräsentation.

¹Eine Möglichkeit zum Angeben von Gruppenwünschen kann gewährt werden, ein Anspruch besteht nicht.

3.1. Implementierung

Die in der Aufgabenstellung spezifizierten Algorithmen und Funktionalitäten müssen gemäß den dort genannten Anforderungen implementiert werden.

In jedem Fall muss zusätzlich zur Hauptimplementierung mindestens eine weitere Vergleichsimplementierung mit abgegeben werden, die geeignet ist, die Qualität der Hauptimplementierung zu evaluieren; wenn nötig sind auch mehrere Vergleichsimplementierungen zu erstellen.

Das Rahmenprogramm ist so zu erstellen, dass es sich genau an die Vorgaben der Aufgabenstellung hält. Die vorgegebenen Ein- und Ausgabeformate sind genau einzuhalten und fehlerhafte Eingaben sind mit einer sinnvollen Fehlermeldung und einem passenden Exit-Code zu behandeln. Für Fehlermeldungen ist `std::err` zu benutzen. Nichteinhalten der Angaben aus der Aufgabenstellung kann hier zu deutlichen Punktabzügen oder sogar zum Nichtbestehen führen.

Zusammen mit dem Programm ist ein `Makefile` abzugeben, das durch den Aufruf von `make`² genau eine ausführbare Binärdatei erzeugt. Diese Binärdatei muss alle abgegebenen Implementierungen enthalten, für die Performanzmessungen geeignet sein und die erforderlichen Test- bzw. Messroutinen bereitstellen.

Sehr wichtig ist das sinnvolle Strukturieren und Kommentieren des Codes. Unzureichende oder nichtssagende Dokumentation sowie eine schwer nachvollziehbare Struktur werden die Benotung negativ beeinflussen. Unverständlicher Code, der nicht oder nur schwer nachvollziehbar ist, kann nicht oder nur mit deutlichen Punktabzügen bewertet werden.

3.2. Projektbericht

Zusammen mit dem Programm ist ein technischer Projektbericht im Markdown-Format abzugeben (max. 800 Wörter, grundsätzlich in deutscher Sprache). Der Bericht soll nach einer kurzen Einleitung den Fokus auf die Dokumentation der Optimierungen und die anschließende Evaluation legen. Hierbei sind die gewählten Lösungsansätze, die verwendete Benchmarking-Methodik sowie die Testumgebung schlüssig darzustellen. Ein wesentlicher Teil des Berichts ist die Performanz- und Korrektheitsanalyse, in der die erzielten Ergebnisse aufbereitet und kritisch interpretiert werden. Abschließend muss für jedes Gruppenmitglied ein kurzer Abschnitt verfasst werden, der den individuellen Anteil an der Projektarbeit explizit beschreibt. Der Bericht dient als primäre Grundlage für die Bewertung im Bereich Analyse.

3.3. Projektpräsentation

Im Anschluss an das Praktikum muss jede Gruppe einen Projektvortrag von 15 Minuten halten (hartes Zeitlimit!). In diesem sollen die wichtigen Aspekte des Projekts vorgestellt werden.

Folgende Bereiche sind in einem sinnvoll gewählten Umfang zu behandeln.

²Aufruf ohne zusätzliche Targets, also nur `'make'`.

1. *Problemstellung*

In diesem einleitenden Teil soll die Problemstellung eingeführt, beschrieben, gegebenenfalls analysiert und genauer spezifiziert werden.

2. *Lösungsansätze und Optimierungen*

Der gewählte Lösungsansatz soll klar und nachvollziehbar beschrieben sowie analysiert werden. Wichtige Entscheidungen werden diskutiert und begründet, umgesetzte Optimierungen werden erläutert. Wo relevant beinhaltet dies eine Gegenüberstellung möglicher Lösungsalternativen und Überlegungen zu möglichen weiteren Optimierungen.

3. *Korrektheit bzw. Genauigkeit (je nach Aufgabe)*

Die Korrektheit bzw. Genauigkeit (jeweils wo angemessen) *der Implementierung und des Ansatzes* soll gezeigt und bewertet werden. Nach Möglichkeit sind hierfür auch automatisierte Tests zu implementieren und abzugeben. Außerdem sollen *sinnvolle und repräsentative Beispiele* für Eingaben und die dazu berechneten Ergebnisse gezeigt werden.

4. *Performanzanalyse*

In diesem Teil soll die Performanz der Implementierung analysiert und bewertet werden. Hierzu sind geeignete Methoden zu wählen, beispielsweise Zeitmessungen. Bei Zeitmessungen ist mindestens eine weitere zur Evaluierung der Implementierung geeignete Vergleichsimplementierung heranzuziehen und diese auf verschiedenen repräsentativen Eingaben auszuführen. Die gewonnenen Ergebnisse sind zu bewerten und Ursachen für beobachtete Performanzunterschiede zu benennen.

5. *Zusammenfassung und Ausblick*

Neben einer kurzen Zusammenfassung des umgesetzten Projektes kann dieser Abschnitt einen kurzen Ausblick enthalten. In diesem wird beurteilt, wo zusätzliches Potential für Verbesserungen existiert, ob rückblickend eine andere Lösungsalternative besser wäre.

Es wird erwartet, dass der Vortrag in ungefähr gleichen Teilen von allen Gruppenmitgliedern gehalten wird. Der Vortragstermin wird durch die Praktikumsleitung zugeteilt. Die Folien der Abschlusspräsentation sind zusammen mit den anderen Dokumenten als PDF *am Abgabetermin des Projekts* mit abzugeben. Die Folien müssen nummeriert sein, ein Folien-Format von 16:9 haben mit einem sinnvollen Kontrast. Es ist nicht möglich, während des Vortrags andere Folien zu verwenden.

Im Rahmen des Vortrags können jedem Gruppenmitglied zu jedem Teil des Projekts Fragen gestellt werden. Zusätzlich werden die Praktikumsleitung, Lehrstuhlmitarbeitenden und die Tutoren die Möglichkeit haben, den Teilnehmenden Fragen zu den im Praktikum erlernten Inhalten, den verwendeten Hardware- und Software-Komponenten und der relevanten Theorie zu stellen.

4. Bewertung

Es werden nur Studierende benotet, deren Gruppe die Präsentation und den Projekt-Code in den erwarteten Formaten eingereicht und die Projektpräsentation gehalten haben. Die Tutoren und Projektbetreuer bewerten die Ergebnisse nach den in den Bewertungsbögen aufgeführten Kriterien.

4.1. Zusatzleistung für 8 ECTS

Bei Studierenden, die 8 ECTS benötigen (siehe Abschnitt [2.6](#)) und die Zusatzleistung nicht erbracht haben oder die Anforderungen nicht vollständig erfüllt haben, wird das Praktikum unabhängig von der sonstigen Projektarbeit mit der Note 5.0 bewertet. Im Übrigen hat die Erbringung der Zusatzleistung keinen Einfluss auf die Bewertung.

4.2. Notenbildung

Aus den Bewertungskriterien ergeben sich Einzelbewertungen für Funktionalität, Optimierungen und Analyse, und Vortrag. Es müssen für *alle* Einzelbewertungen mindestens 40% der Punkte erreicht werden, andernfalls wird das Praktikum direkt mit der Note 5.0 bewertet. Die Gesamtpunktzahl ergibt sich aus der Summe dieser Einzelbewertungen. Der Notenbonus verbessert die Note um eine Notenstufe (0.3/0.4); eine Verbesserung der Noten 1.0, 4.3, 4.7 und 5.0 ist dabei ausgeschlossen.

4.3. Einsicht

Bei der Einsicht können Studierende sich über die Zusammensetzung der Note informieren. Das Ziel der Einsicht ist die Nachvollziehbarkeit der Bewertung für die Studierenden. Mit Freigabe der Bewertung werden die Formalitäten der Einsicht bekannt gegeben. Beanstandungen sind spätestens am letzten Einsichtstag einzureichen und werden im Anschluss überprüft. Eine Antwort durch die Praktikumsleitung auf die Beanstandung kann auch per E-Mail erfolgen. Durch eine etwaige Nachkorrektur kann es auch zu einer Verschlechterung der Bewertung kommen.

5. Sonderfälle

5.1. Gruppen aus weniger als drei Personen

Bei der Gruppenbildung ist darauf zu achten, dass keine Einzelpersonen übrig bleiben und es maximal viele Dreiergruppen gibt. Das bedeutet, dass eine Einzelperson und eine Dreiergruppe sich zusammen in zwei Zweiergruppen teilen.

Falls ein Gruppenmitglied während des Projekts die Gruppe verlässt oder sich trotz wiederholter Bemühungen der übrigen Gruppe nicht oder nur sehr geringfügig an der Projektarbeit beteiligt, sind der betreuende Tutor und die Praktikumsleitung *frühzeitig*

A. Bewertungskriterien

Folgende Listen beschreiben die Verteilung der Punkte auf die Projektbestandteile.

Funktionalität (20 Pkt.)

Grundfunktionalität (8 Punkte)

- Erfolgreicher Build-Prozess und korrekte Programmausführung
- Korrekte Verarbeitung der bereitgestellten simplen Eingabedaten
- Einhaltung der spezifizierten Ausgabeformate
- Grundlegende Übereinstimmung mit dem Referenzverhalten

Erweiterte Funktionalität (12 Punkte)

- Vollständigkeit aller geforderten Programmfeatures
- Randfälle gut abgedeckt, sinnvolle Fehlermeldungen
- Aufgabenspezifische Kriterien erfüllt
- Korrekte Speicherverwaltung
- Keine Segmentation Faults
- Keine unsicheren Funktionen verwendet
- Korrekte Ergebnisse aller Implementierungen
- Kein redundanter Code
- Einhaltung von Coding-Standards
- etc.

Formalitäten (Abzug)

- (-20) Abgabe kompiliert nicht/unfunktional, kein/unfunktionales Makefile, mehrere Binaries, mehrere Code-Versionen in Abgabe etc., formale Fehler die schon die Grundfunktionalität beeinträchtigen
- (-10) Aufgabenstellung grob verletzt
- (Je nach Schwere) Andere formale Fehler, die die Funktionalität nicht beeinträchtigen aber die Korrektur behindern

Hinweis:

- Unverständlicher oder formal völlig unzureichender Code kann nicht korrigiert werden.
 - Punkte können nur gegeben werden, wenn eine Gruppe in Code und Vortrag zeigt, dass sie die entsprechenden Inhalte verstanden hat. Es ist nicht möglich, Punkte für die Funktionalität zu erhalten, wenn die Gruppe im Vortrag nicht zeigen kann, dass sie die eigene Implementierung verstanden hat.
-

Optimierung und Analyse (20 Pkt.)**Optimierungen (8 Punkte)**

- Anwendung des in der Übungsphase Erlernten Wissen über Optimierungsmöglichkeiten
- Implementierung fortgeschrittener algorithmischer Optimierungen
- Angemessener und systematischer Einsatz von Vektorisierung (SIMD)
- Berücksichtigung hardwarenaher Aspekte wie Cache-Effizienz, Mikrooptimierungen, etc.
- Angemessener und konsistenter Optimierungsgrad über alle Programmkomponenten hinweg
- Effiziente Gestaltung der Ein- und Ausgabeoperationen
- etc.

Benchmarks (5 Punkte)

- Grundsätzlich sinnvolle und nachvollziehbare Methodik zur Performanzmessung
- Technisch korrekte Messmethodik zur Laufzeitbestimmung
- Auswahl und Implementierung geeigneter Referenzwerte zur Evaluation
- Systematische Durchführung aussagekräftiger Testfälle
- Vollständige Dokumentation der Messparameter und Testumgebung
- etc.

Performanzanalyse (4 Punkte)

- Wissenschaftliche Aufbereitung der Messdaten in geeigneter graphischer Form
- Nachvollziehbare Interpretation der Ergebnisse und Identifikation von Flaschenhälsen
- Kritische Reflexion der erreichten Speedup-Faktoren

Korrektheitsanalyse (3 Punkte)

- Auswahl einer sinnvollen Validierungsmethodik und Erläuterung dieser
- Methodische Überprüfung funktionaler Randfälle
- Nachvollziehbarer Nachweis der Ergebnisgenauigkeit

Formalitäten (Abzug)

- (-10) Methodische Mängel führen zu unbrauchbaren Ergebnissen
- (Je nach Schwere) Formale Mängel im Kurzbericht oder betreffenden Codeteilen, die die Korrektur behindern

Hinweis:

- Es ist nicht möglich, Punkte für die Optimierung und Analyse zu erhalten, wenn die Gruppe im Vortrag nicht zeigen kann, dass sie die eigenen Optimierungen und die eigene Analyse verstanden hat.
 - Unverständliche oder formal völlig unzureichende Optimierungen und Analysen können nicht korrigiert werden.
-

Abschlussvortrag (20 Pkt.)**Problemstellung (1 Punkte)**

- Wissenschaftliche Einordnung und Erläuterung der Aufgabenstellung
- Visuelle Veranschaulichung

Lösungsansätze und Optimierungen (4 Punkte)

- Darstellung der algorithmischen und technischen Kernkonzepte
- Abhandlung der Vergleichsimplementierungen
- Erläuterung der Optimierungsstrategien (z. B. SIMD, Farbberechnung)
- Nutzung angemessener Visualisierungen zur Veranschaulichung der Lösungsansätze und Optimierungen
- Methodik zur Behandlung von relevanten Randfällen

Korrektheitsanalyse (1 Punkte)

- Erläuterung der Methodik zur Überprüfung der Ergebnisgenauigkeit
- Nutzung angemessener Visualisierungen zur Veranschaulichung der Korrektheitsanalyse

Performanzergebnisse (4 Punkte)

- Darstellung der Benchmarking-Methodik und Testumgebung
- Graphische Aufbereitung und Interpretation der Messdaten
- Analyse des Laufzeitverhaltens und der erzielten Beschleunigung

Sonstige Foliengestaltung (2 Punkte)

- Verständlichkeit, Unterstützung des Vortrags, Rechtschreibung, Lesbarkeit, Diagramme etc.

Fachliches Verständnis (8 Punkte)

- Beantwortung vertiefender Fragen zu Theorie und Projekt
- Präzise Argumentation und fachliche Durchdringung des eigenen Beitrags
- Angemessener Vortragsstil und aktive Mitwirkung an der Diskussion
- etc.

Formalitäten (Abzug)

- Abgabeformalitäten nicht eingehalten
 - Fehlende inhaltliche Eigenleistung einer Person
 - etc.
-